SENTINEL - Local AI Code Review and Engineering Intelligence Platform

A privacy-first system for evidence-backed repository analysis and guarded AI assistance

ABSTRACT

Modern software repositories are difficult to understand because source code, dependencies, architecture, security signals, and review history are spread across separate tools. Generative AI can accelerate review, but its output may be incorrect, unverifiable, or unsafe to apply. SENTINEL addresses this problem through a local-first engineering intelligence platform that combines deterministic static analysis with optional local AI assistance while preserving a clear boundary between evidence and model opinion.

The platform indexes a permitted repository directory without importing or executing repository code. It detects files, languages, frameworks, package managers, dependencies, tests, source entities, and content fingerprints. Python abstract syntax trees and JavaScript, TypeScript, and TSX syntax trees support a bounded ruleset of security, quality, and performance findings. Each finding records its file, line range, symbol, risk basis, and structural evidence. A persistent repository graph and local embedding index add relationship exploration and semantic search.

Optional AI workflows use installed GGUF models through a loopback Ollama service. Context is bounded, source fingerprints are rechecked, detected secret assignments are redacted, and model responses are treated as untrusted data. Explanations must satisfy a schema and cite supplied evidence. Python fix and pytest proposals pass syntax, import, selected-symbol, exact-diff, model digest, review-hash, and source-fingerprint gates. Approved tests can run only in a pinned Docker sandbox with no network, a read-only root filesystem, a non-root user, resource limits, and no host-execution fallback. Applying or rolling back a change requires explicit approval and atomic fingerprint-checked file operations.

SENTINEL was implemented incrementally across repository indexing, deterministic analysis, local reasoning, graph and semantic search, and reviewed changes. The verified build passes 117 automated tests together with Python linting, TypeScript validation, database migration checks, production builds, a live API smoke scan, and browser workflows. The result is not a claim of complete security or autonomous remediation. It is an evidence-centered review environment that improves repository understanding while making uncertainty, coverage, approval, and execution boundaries visible. Future work includes a safe local GitHub importer, report export, command-line and desktop clients, broader analysis, scan-to-scan trends, and a hosted edition based on GitHub Apps, durable queues, managed PostgreSQL, tenant isolation, and disposable analysis workers.

Keywords: static analysis; local AI; code review; repository intelligence; semantic search; secure sandboxing; software engineering.